Dynamic programming on paths

Paths on a line and paths through a grid

FRE6883 – Financial Computation – Week 5

A DP solution starts with four decisions

1. **State:** What does $dp[i]$ mean exactly?
2. **Transition:** Which smaller states can lead to i?
3. **Base case:** What is true at the starting position?
4. **Order:** Are all predecessors ready before $dp[i]$?

The same index i can store a *count* or a *minimum cost*. The question determines the value stored in the vector.

Easy | Count ways to climb stairs

Problem 1 LeetCode 70 pattern

Start at step 0. A move climbs 1 or 2 steps. Count the ways to reach step n.

- $0 \leq n \leq 40$. Different move sequences count separately.
- Examples: $n = 0 \to 1$; $n = 4 \to 5$.

Before coding: List every route to step 3. What can the final move be?

The last move gives the recurrence

$dp[i]$ = number of ways to reach exactly step i.

$$dp[i] = dp[i-1] + dp[i-2] \quad (i \geq 2), \qquad dp[0] = 1, \quad dp[1] = 1.$$

i	0	1	2	3	4	5
$dp[i]$	1	1	2	3	5	8

$dp[0] = 1$ counts the empty sequence. It makes the recurrence work at the first steps. Time $O(n)$, space $O(n)$.

Easy | Complete the first DP function

Complete the small gaps

`std::vector`, indexing, addition

```
int countStairs(int n) {
    std::vector<int> dp(________, 0);
    dp[0] = ________;
    if (n >= 1) dp[1] = ________;
    for (int i = ____; i <= n; ++i) {
        dp[i] = ____________________;
    }
    return ________;
}
```

What does the function return when $n = 0$? Why is `n + 1` the vector size?

Easy | Correction and invariant

Every entry is ready before the next one

$O(n)$ time, $O(n)$ space

```
int countStairs(int n) {
    std::vector<int> dp(n + 1, 0);
    dp[0] = 1;
    if (n >= 1) dp[1] = 1;
    for (int i = 2; i <= n; ++i) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}
```

Invariant: after iteration i, $dp[0], \ldots, dp[i]$ contain the correct counts.

The STL stores states without manual allocation

- `#include <vector>` provides the container used by both counting functions.
- `std::vector<int> dp(n + 1, 0)` creates indices 0 through n, initially zero.
- `dp[i]` reads or writes the state for position i.
- We use 32-bit `int` values; the exercise bounds keep every count and cost in range.
- `#include <climits>` provides `INT_MAX`, the largest `int`, used to initialize minimum costs.

Check: if the vector had size n, which access would be invalid?

Problem 2 Generalize the transition

Moves may have length 1, 2, or 3. A path cannot land on a forbidden position. Count paths from 0 to n.

- Input: $0 \leq n \leq 30$ and a list of forbidden positions in $1, \ldots, n$.
- Example: $n = 4$, forbidden $\{2\} \to 3$ paths.
- Return 0 when the target n itself is forbidden.

Why the stair recurrence is no longer enough

The old formula only looks back 1 or 2 positions. A jump of length 3 adds another predecessor, and a forbidden landing must contribute zero.

$$dp[i] = \begin{cases} 0, & i \text{ is forbidden,} \\ \sum_{d=1}^{3} dp[i-d], & \text{otherwise; omit } i-d<0. \end{cases}$$

i	0	1	2	3	4
Forbidden?	no	no	yes	no	no
$dp[i]$	1	1	0	2	3

Complete the flag and base case

The flags encode forbidden landings

```
int countPaths(int n,
               const std::vector<int>& forbidden) {
    std::vector<bool> blocked(n + 1, false);
    for (int x : forbidden) {
        if (1 <= x && x <= n) blocked[x] = ____;
    }
    std::vector<int> dp(n + 1, 0);
    dp[0] = ____;
    // Continue on the next slide.
```

Why is the empty path counted once? What value will a forbidden position keep?

Complete the loop bounds and update

Follow the recurrence

```
    for (int i = ____; i <= ____; ++i) {
        if (__________) continue;
        for (int step = ____; step <= ____; ++step) {
            int previous = __________;
            if (previous >= ____) {
                dp[i] = ________ + ________;
            }
        }
    }
    return dp[n];
}
```

For $i = 3$ and blocked step 2, which three previous entries are added?

The flags represent inaccessible landings

$O(n + m)$ time, $O(n)$ space

```
int countPaths(int n,
               const std::vector<int>& forbidden) {
    std::vector<bool> blocked(n + 1, false);
    for (int x : forbidden) {
        if (1 <= x && x <= n) blocked[x] = true;
    }
    std::vector<int> dp(n + 1, 0);
    dp[0] = 1;
    // Transition on the next slide.
```

m is the length of `forbidden`; out-of-range entries are ignored defensively.

Skip a blocked target before adding paths

$dp[0] = 1$ still matters

```
    for (int i = 1; i <= n; ++i) {
        if (blocked[i]) continue;
        for (int step = 1; step <= 3; ++step) {
            int previous = i - step;
            if (previous >= 0) {
                dp[i] = dp[i] + dp[previous];
            }
        }
    }
    return dp[n];
}
```

Invariant: $dp[i]$ counts valid paths ending at i; a forbidden state stays zero.

Intermediate | Read one correction step by step

With $n = 4$ and blocked position 2, the code leaves $dp[2] = 0$. At position 3, the three previous states contribute:

$$dp[3] = dp[2] + dp[1] + dp[0] = 0 + 1 + 1 = 2.$$

At position 4, $dp[4] = dp[3] + dp[2] + dp[1] = 2 + 0 + 1 = 3$.

`continue` leaves a blocked destination at zero; later positions can still jump over it. The bounds check prevents reading a negative index.

Problem 3 A guided minimum-cost DP

There are n stones with heights $h_0, \ldots, h_{n-1}$. From stone j, jump to any i with $1 \le i - j \le K$. The jump costs

$$|h_i - h_j| + c(i-j)^2.$$

Find the minimum total cost from stone 0 to stone $n-1$. $1 \le n \le 2000$, $1 \le K \le n$, $|h_i| \le 1000$, $0 \le c \le 100$. Example: $h = [10, 30, 40, 20]$, $K = 3$, $c = 1 \to 19$.

The old count is the wrong state value

We still move left to right, but adding predecessor counts cannot compare routes with different costs. Now $dp[i]$ means the *minimum cost* to reach stone i.

$$dp[0] = 0, \qquad dp[i] = \min_{\max(0,i-K)\le j<i} \big(dp[j] + |h_i - h_j| + c(i-j)^2\big).$$

The last jump comes from one of up to K stones. Enumerate every valid predecessor, add its jump cost, and keep the minimum.

Trace the optimal cost before coding

For $h = [10, 30, 40, 20]$, $K = 3$, $c = 1$:

Stone i	Candidate costs from earlier stones	$dp[i]$
0	Start	0
1	$0 + 20 + 1$	21
2	$0 + 30 + 4$, $21 + 10 + 1$	32
3	$0 + 10 + 9$, $21 + 10 + 4$, $32 + 20 + 1$	19

Subtlety: the best route to stone 3 jumps directly from stone 0. A locally short jump is not necessarily cheaper.

Hard | Guided code, initialize the state

Complete three values

The signature and type are provided

```
int minFatigue(const std::vector<int>& h,
               int K, int c) {
    const int n = h.size();
    std::vector<int> dp(n, ________);
    dp[____] = ________;
    // Compute the remaining states on the next slide.
```

What does $dp[i]$ measure? Why must every unknown minimum start at `INT_MAX`?

Hard | Guided code, examine last jumps

Choose predecessor bounds

Complete the cost comparison

```
    for (int i = ____; i < ____; ++i) {
        for (int j = std::max(____, i - ____);
             j < ____; ++j) {
            int d = ________;
            int cost = std::abs(h[i] - h[j])
                           + __________;
            dp[i] = std::min(________,
                             ________ + cost);
        }
    }
    return ________;
}
```

Match `j` to the last stone before `i`. Then test the trace ending in cost 19.

Hard | Test the boundary cases

Input	Expected result
One stone: $h = [7]$	0
$h = [10, 30, 40, 20], K = 3, c = 1$	19
$h = [5, 5, 5], K = 1, c = 2$	4
$h = [5, 5, 5], K = 2, c = 2$	4

For the last row, two jumps of length 1 cost $2 + 2 = 4$; one jump of length 2 costs 8.

Hard | Correction, state and bounds

```cpp
#include <algorithm>
#include <cstdlib>
#include <climits>
#include <vector>

int minFatigue(const std::vector<int>& h,
               int K, int c) {
    const int n = h.size();
    std::vector<int> dp(n, INT_MAX);
    dp[0] = 0;
    // Complete the recurrence on the next slide.
```

`INT_MAX` marks an unknown minimum. Every predecessor is already reachable, so it is never added to a cost.

Hard | Correction, try every last jump

```
    for (int i = 1; i < n; ++i) {
        for (int j = std::max(0, i - K); j < i; ++j) {
            int d = i - j;
            int cost = std::abs(h[i] - h[j])
                            + c * d * d;
            dp[i] = std::min(dp[i], dp[j] + cost);
        }
    }
    return dp[n - 1];
}
```

Invariant: when iteration i ends, $dp[i]$ is the cheapest route to stone i. Time $O(nK)$, space $O(n)$.

Hard | Why the corrected code works

1. `dp[0] = 0`: starting on the first stone has no jump cost.
2. `j = max(0, i - K)` through `i - 1`: these are exactly the legal last-jump origins.
3. `dp[j] + cost`: an optimal route to j, followed by that last jump.
4. `std::min`: keep the cheapest candidate for i.

Because every predecessor $j < i$ was computed earlier, the loop produces the correct minimum at i. For one stone, the loop is skipped and the answer is zero.

A grid needs two coordinates for each state

On a line, $dp[i]$ describes one destination. How should we describe a destination in a grid?

A grid needs two coordinates for each state

On a line, $dp[i]$ describes one destination. How should we describe a destination in a grid?

In a grid, $dp[r][c]$ describes the destination at row r, column c.

A grid needs two coordinates for each state

On a line, $dp[i]$ describes one destination. How should we describe a destination in a grid?

In a grid, $dp[r][c]$ describes the destination at row r, column c.

If moves go only right or down, the last move into (r, c) came from $(r-1, c)$ or $(r, c-1)$.

C++ stores the table as `std::vector<std::vector<int>>`. The outer vector selects a row; the inner vector selects a column.

Grid paths | An obstacle removes a destination

Count routes from the top-left to the bottom-right, moving only right or down. A # cell cannot be visited.

Grid

```
. . .
. # .
. . .
```

Grid paths | An obstacle removes a destination

Count routes from the top-left to the bottom-right, moving only right or down. A # cell cannot be visited.

Grid

```
. . .
. # .
. . .
```

Number of paths

1	1	1
1	0	1
1	1	**2**

Grid paths | An obstacle removes a destination

Count routes from the top-left to the bottom-right, moving only right or down. A # cell cannot be visited.

Grid	Number of paths
. . .	1 1 1
. # .	1 0 1
. . .	1 1 **2**

$dp[0][0] = 1$ if the start is open. A blocked cell has value 0; otherwise add top and left when they exist.

Assume a rectangular grid with $1 \leq n, m \leq 15$ and cells `.` or `#`.

Grid paths | Complete the two loops

Decide the row and column bounds

Use the two predecessors

```
// Inside countGridPaths(g): n rows, m columns.
std::vector<std::vector<int>> dp(
    n, std::vector<int>(m, 0));
if (g[0][0] == '#') return 0;
dp[0][0] = ____;
for (int r = ____; r < ____; ++r) {
    for (int c = ____; c < ____; ++c) {
        if (__________) continue;
        if (r > 0) dp[r][c] += __________;
        if (c > 0) dp[r][c] += __________;
    }
}
return dp[n - 1][m - 1];
```

Why does the loop still visit $(0, 0)$ without changing its initial value?

Grid paths | Correction

```cpp
int countGridPaths(const std::vector<std::string>& g) {
    int n = g.size();
    int m = g[0].size();
    std::vector<std::vector<int>> dp(
        n, std::vector<int>(m, 0));
    if (g[0][0] == '#') return 0;
    dp[0][0] = 1;
    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < m; ++c) {
            if (g[r][c] == '#') continue;
            if (r > 0) dp[r][c] += dp[r - 1][c];
            if (c > 0) dp[r][c] += dp[r][c - 1];
        }
    }
    return dp[n - 1][m - 1];
}
```

Grid paths | Why the correction works

- $dp[r][c]$ counts routes ending at (r, c).
- A blocked destination contributes zero; the loop leaves its initialized value unchanged.
- Top and left are already computed in row-major order.
- If the start is blocked, return zero before setting $dp[0][0]$.

For an $n \times m$ grid, time is $O(nm)$ and table space is $O(nm)$. The same recurrence handles a blocked finish.

Minimum grid sum | Keep the cheapest route

Each cell now has a nonnegative cost. Find the minimum sum of cells on a right/down path, including both endpoints.

Cell costs

1	3	1
1	5	1
4	2	1

Minimum grid sum | Keep the cheapest route

Each cell now has a nonnegative cost. Find the minimum sum of cells on a right/down path, including both endpoints.

Cell costs

1	3	1
1	5	1
4	2	1

Minimum sums

1	4	5
2	7	6
6	8	**7**

Minimum grid sum | Keep the cheapest route

Each cell now has a nonnegative cost. Find the minimum sum of cells on a right/down path, including both endpoints.

Cell costs

1	3	1
1	5	1
4	2	1

Minimum sums

1	4	5
2	7	6
6	8	**7**

$dp[0][0] = a[0][0]$. Elsewhere, add the cell cost to the cheaper valid predecessor. Counting paths cannot tell us which route is cheapest.

Assume $1 \le n, m \le 200$ and $0 \le a[r][c] \le 10^6$.

Minimum grid sum | Choose the bounds

Reuse the grid traversal

Change the stored value

```
// Inside minGridSum(a): n rows, m columns.
std::vector<std::vector<int>> dp(
    n, std::vector<int>(m, INT_MAX));
dp[0][0] = __________;
for (int r = ____; r < ____; ++r) {
    for (int c = ____; c < ____; ++c) {
        if (r == 0 && c == 0) continue;
        int best = ________;
        if (r > 0) best = std::min(best, dp[r - 1][c]);
        if (c > 0) best = std::min(best, dp[r][c - 1]);
        dp[r][c] = __________ + __________;
    }
}
return dp[____][____];
```

Why does every cell except the start have a computed predecessor, making `best` finite?

Minimum grid sum | Correction

```cpp
int minGridSum(const std::vector<std::vector<int>>& a) {
    int n = a.size();
    int m = a[0].size();
    std::vector<std::vector<int>> dp(
        n, std::vector<int>(m, INT_MAX));
    dp[0][0] = a[0][0];
    for (int r = 0; r < n; ++r) {
        for (int c = 0; c < m; ++c) {
            if (r == 0 && c == 0) continue;
            int best = INT_MAX;
            if (r > 0) best = std::min(best, dp[r-1][c]);
            if (c > 0) best = std::min(best, dp[r][c-1]);
            dp[r][c] = best + a[r][c];
        }
    }
    return dp[n-1][m-1];
}
```

Minimum grid sum | Read one update

At cell $(1, 1)$ in the example, the top route costs 4 and the left route costs 2. The cell itself costs 5:

Minimum grid sum | Read one update

At cell $(1, 1)$ in the example, the top route costs 4 and the left route costs 2. The cell itself costs 5:

$$dp[1][1] = 5 + \min(4, 2) = 7.$$

- On the top row, only the left predecessor exists.
- In the first column, only the top predecessor exists.
- The start is initialized separately, so every other cell has a valid predecessor.

Time $O(nm)$, table space $O(nm)$; a one-cell grid returns its cell cost.

One last move, five path problems

Question	State value	Transition
Stairs	Number of paths	Sum 2 predecessors
Forbidden steps	Number of valid paths	Sum 3 predecessors, or zero
Fatigue jumps	Minimum cost	Minimum over up to K predecessors
Grid obstacles	Number of paths	Sum top and left, or zero
Grid costs	Minimum sum	Cell cost plus cheaper predecessor

The stored value and number of indices change. The method remains: state, last move, base case, computation order.

Exit ticket | Add diagonal moves to a grid

On a grid with $1 \leq n, m \leq 10$, suppose an open cell can also be reached diagonally from $(r-1, c-1)$. How does the grid-path recurrence change?

Exit ticket | Add diagonal moves to a grid

On a grid with $1 \le n, m \le 10$, suppose an open cell can also be reached diagonally from $(r-1, c-1)$. How does the grid-path recurrence change? **Correction:** keep $dp[0][0] = 1$ for an open start. For any other open cell,

$$dp[r][c] = dp[r-1][c] + dp[r][c-1] + dp[r-1][c-1],$$

omitting predecessors outside the grid. A blocked cell still has value zero.